Describing a Table

Reading a delimited file into one array per column, NaN and the tests it needs, counting the categories in a column, mean, median, standard deviation and quantiles, pairwise against listwise deletion, grouped summaries, and a bar chart.

01 Reading a delimited file
02 Missing values and NaN
03 Counting rows and categories
04 Summary statistics
05 Pairwise and listwise deletion
06 Grouped summaries
07 Bar charts
08 Loop forms and array equivalents

Scope. Arrays, slicing, masks and reductions are assumed. What is new is the shape of the data: a table of rows and columns, where each column holds a different kind of value and not every field is filled. Everything below is about loading such a table and saying what is in it, which kinds of row it holds, how its numbers are spread, and how that differs between groups.

After reading it you should be able to do the following without looking anything up.

- Read a delimited file into one array per column, each with a dtype of its own, or into one array for the whole table with the categories coded as integers.
- Represent a missing number, and say what a sum or a mean over it returns.
- Count how many rows fall into each category of a categorical column, including a category with no rows.
- Compute mean, median, standard deviation, extremes and quantiles, and say which of them a single extreme value moves.
- Reduce every column of a table in one call with `axis=0`, and sort a column that has gaps in it without the NaNs ending up where the largest values should be.
- Skip missing values pairwise rather than dropping whole rows, and state the difference it makes.
- Summarise a numeric column within each category of a categorical one, and read the same counts off a chart.

Every example assumes `import csv`, `import io`, `import numpy as np` and `import matplotlib.pyplot as plt`. The running example is an export of 9000 customer records with seven columns: four numeric (`age`, `income`, `tenure_months`, `score`) and three categorical (`region`, `plan`, `channel`).

01

Reading a delimited file

Every field in a CSV file is text. Nothing in the file records what type a column is or what encoding it was written in, and the rules for splitting a line into fields are a convention rather than a standard.

```
age,income,tenure_months,score,region,plan,channel
47,,20,33.8,Southgate,legacy,shop
57,18581.51,19,45.8,Midland,basic,online
```

```
open("register.csv", newline="", encoding="utf-8").readline()
# -> 'age,income,tenure_months,score,region,plan,channel\r\n'
# without newline="" the \r\n is translated to \n before you see it
```

The columns do not share a type: three of them are text and four are numbers. The second row has nothing between the first and second commas, which is what a missing value looks like in a CSV export.

WHAT THE FORMAT PROMISES

There is no CSV standard that every writer follows. RFC 4180 describes the common form, and most tools approximate it:

- Fields are separated by a delimiter. Usually a comma, but a semicolon where the comma is the decimal separator, and a tab in a file called TSV.
- A field containing the delimiter, a quotation mark or a newline is wrapped in quotation marks.
- A quotation mark inside a quoted field is written twice.
- The first line is a header by convention. Nothing in the file marks it as one.

None of that is enforced. A file can change its column count halfway down, quote inconsistently, or have a header that does not match what follows, and still be a CSV.

Quoting decides how the file has to be read. Splitting on the delimiter also splits the fields that contain one:

```
line = 'Smith, John,42,"Oxford, Oxfordshire"'

line.split(",")          # -> ['Smith', ' John', '42', '"Oxford', ' Oxfordshire"']   5 fields
next(csv.reader([line])) # -> ['Smith', ' John', '42', 'Oxford, Oxfordshire']        4 fields
```

The other two rules have the same consequence. A doubled quotation mark is one character, not two, and a newline inside a quoted field does not end the record, so a file of 9000 records can have more than 9001 lines in it.

```
doubled = 'id,note\n7,"she said ""no"" twice"\n'
list(csv.reader(io.StringIO(doubled)))[1]    # -> ['7', 'she said "no" twice']

wrapped = 'id,note\n7,"line one\nline two"\n'
len(list(csv.reader(io.StringIO(wrapped))))  # -> 2 records
len(wrapped.strip().split("\n"))             # -> 3 lines
```

The csv module implements all three rules, which is why a CSV is read through it rather than through `str.split`.

WHAT THE FILE DOES NOT SAY

Three things have to come from outside the file, and getting any of them wrong is silent.

- **The types.** Every field is text. That `47` is an age and `33.8` a score is a decision made by the reader.
- **The encoding.** Nothing in a CSV records whether it is UTF-8 or something older, so a name with an accent in it either survives or turns into mojibake depending on what the reader assumed.
- **What counts as missing.** An empty field is one convention. `NA`, `NULL`, `-999` and `-1` are others, and a reader that does not know about them will load a sentinel as a number and average it.

MIXED COLUMN TYPES

`np.loadtxt` and `np.genfromtxt` read a delimited file straight into an array. They fit a file that is numeric throughout. This one is not: a single array has a single dtype, so a table with text and numbers in it has to become either several arrays or one array of strings, and an array of strings cannot be averaged.

The reliable route is to read the rows as text first and convert each column separately. `csv.DictReader` yields one dictionary per row, keyed by the header line, so columns are named rather than numbered.

```
with open("register.csv", newline="", encoding="utf-8") as f:
    records = list(csv.DictReader(f))

len(records)            # -> 9000
records[0]["age"]       # -> '47'      a string, not an int
records[1]["income"]    # -> '18581.51' also a string
records[0]["income"]    # -> ''        the empty field from the line above
```

The csv module does its own line-ending handling, so its documentation asks for `newline=""` on any file passed to it. This file ends its lines with `\r\n`, which a plain text-mode read translates to `\n` before the reader sees it. On a file like this one the parsed rows are identical either way; the difference shows in a quoted field that contains a newline of its own, and on writing, where the translation can add a stray `\r`. `encoding="utf-8"` is worth stating rather than inheriting whatever the machine defaults to.

ONE ARRAY PER COLUMN

Each column then becomes its own array with its own dtype. The numeric ones become `float64`, the categorical ones stay as strings.

```
region = np.array([r["region"] for r in records])    # (9000,) dtype '<U9', text
tenure = np.array([float(r["tenure_months"]) for r in records])   # (9000,) float64

tenure.dtype            # -> dtype('float64')
region.dtype            # -> dtype('<U9')   unicode, up to 9 characters
region[:3]              # -> ['Southgate' 'Midland' ...]
```

A string array has a fixed width per element, chosen from the longest string present at creation. Assigning a longer string into it later truncates silently, so build string arrays from the complete data rather than growing them.

The same conversion on `income` does not work, and the reason is the next section.

```
np.array([float(r["income"]) for r in records])
# ValueError: could not convert string to float: ''
```

ONE ARRAY FOR THE WHOLE TABLE

One array per column is not the only option. The other is to make the text columns numeric and keep the whole table in a single array: each category becomes an integer code, and the result is one two-dimensional block of floats, one row per customer and one column per variable.

```
NUMERIC = ["age", "income", "tenure_months", "score"]
CATEGORICAL = ["region", "plan", "channel"]
CATEGORIES = {"region": ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"],
              "plan": ["basic", "standard", "premium", "legacy"],
              "channel": ["shop", "phone", "online"]}
COLUMNS = NUMERIC + CATEGORICAL

CODES = {c: {name: i for i, name in enumerate(CATEGORIES[c])} for c in CATEGORICAL}

rows = []
for r in records:
    numeric = [float(r[c]) if r[c] != "" else np.nan for c in NUMERIC]
    coded = [float(CODES[c][r[c]]) for c in CATEGORICAL]
    rows.append(numeric + coded)

table = np.array(rows, dtype=np.float32)
table.shape             # -> (9000, 7)
table[0]                # -> [47.  nan  20.  33.8  4.  3.  0.]
```

Two things follow. Every per-column statistic becomes one call with `axis=0`, which is section 4. And the table is already the shape a model is fed, a matrix of rows and features.

The cost is that the codes are labels, not quantities. Southgate is 4 and Midland is 2, which does not make Southgate twice Midland, and nothing in the array records that difference.

```
np.nanmean(table, axis=0)[4:]    # -> [2.12 1.16 1.15]   the "mean region", which is nothing
```

Any operation that reads the column as a quantity, a mean or a distance or a linear model, invents an order the categories do not have. The repair is one column per category instead of one code, which is one-hot encoding. Both representations are used: one array per column keeps each column's own dtype and will not let you average a category by accident, and one array for the table buys `axis=0` and the shape a model wants.

PANDAS

The library `pandas` does all of the above in one call: `pd.read_csv("register.csv")` infers a type per column, keeps the names, and represents empty fields as missing automatically. It is the standard tool for this job and worth knowing by name. The point of doing it by hand once is that the decisions it makes for you are exactly the ones that change the answers below.

02

Missing values and NaN

An empty field is not a zero and not an empty string once it reaches a numeric column. It needs a value that arithmetic propagates rather than ignores.

Floating point reserves one: **NaN**, not-a-number, written `np.nan`. It is a float, so it sits in a float array without changing the dtype, and every operation that touches it produces NaN again.

```
def column(name):
    """One numeric column as floats, with empty fields as NaN."""
    return np.array([float(r[name]) if r[name] != "" else np.nan for r in records])

income = column("income")
income.dtype            # -> dtype('float64')  NaN is a float
income[0]               # -> nan
```

NAN PROPAGATES

```
np.nan + 1              # -> nan
income.sum()            # -> nan     one missing value poisons the total
income.mean()           # -> nan
income.max()            # -> nan
```

A summary that silently ignored the gaps would be worse. NaN turns a missing value into a loud one: the answer is NaN until you say what should happen to the gaps.

NAN IS NOT EQUAL TO ITSELF

This follows from the floating-point standard rather than from NumPy. A comparison against NaN is false, including a comparison with another NaN, so `== np.nan` never finds anything.

```
np.nan == np.nan        # -> False
income == np.nan        # -> all False, a useless mask

np.isnan(income)        # (9000,) -> (9000,) bool, True where the value is missing
np.isnan(income).sum()  # -> 823    how many are missing
(~np.isnan(income)).sum()   # -> 8177  how many are present
```

`np.isnan` is the only correct test. It takes a float array and gives a boolean array, which is an ordinary mask from there on.

ZERO IS NOT MISSING

Two shortcuts are common and both change the data. Writing `0.0` for a missing income adds 823 customers who earn nothing, which pulls the mean down and invents a spike at zero in every histogram. Dropping every row that has any gap anywhere is the subject of section 5.

03

Counting rows and categories

Before any statistic, two counts: how many rows there are, and how they are spread across the categories of each categorical column.

```
plan = np.array([r["plan"] for r in records])      # (9000,) text

(plan == "premium").sum()           # -> 1915   one category, counted with a mask
np.unique(plan)                     # -> ['basic' 'legacy' 'premium' 'standard']
names, counts = np.unique(plan, return_counts=True)
list(zip(names, counts))           # -> [('basic', 3048), ('legacy', 1274), ...]
```

`np.unique` returns the categories in sorted order, which is alphabetical for text. That is rarely the order you want to present, and it is never an order the data implies.

CATEGORIES WITH NO ROWS

`np.unique` reports only the categories that occur. If a category is defined by the domain and happens to be absent, it is missing from the output, and a result indexed by position silently shifts. Counting against a fixed list of categories avoids it and makes the zero visible.

```
counts = {name: int((plan == name).sum()) for name in CATEGORIES["plan"]}
counts                  # -> {'basic': 3048, 'standard': 2763, 'premium': 1915, 'legacy':
1274}
sum(counts.values())     # -> 9000, every row accounted for
```

Checking that the counts add up to the number of rows catches a misspelled category, a trailing space in the file, and a category that was not in the list.

MISSING VALUES PER COLUMN

The same counting question applied to the gaps. Each numeric column gets its own count, and the column with no gaps at all is as informative as the others.

```
for name in ["age", "income", "tenure_months", "score"]:
    v = column(name)
    print(name, int(np.isnan(v).sum()))

# age            542
# income         823
# tenure_months    0
# score          361
```

04

Summary statistics

Five numbers describe a numeric column: where it sits, how wide it is, and where it ends. Each answers a different question and each fails differently.

```
score = column("score")
s = score[~np.isnan(score)]         # (9000,) -> (8639,) the present values

s.mean()                # -> 58.73   the balance point
np.median(s)            # -> 58.80   the middle value
s.std()                 # -> 13.20   root mean square distance from the mean
s.min(), s.max()        # -> 9.7, 100.0
np.quantile(s, [0.25, 0.5, 0.75])   # -> [49.8 58.8 67.7]  the quartiles
```

EVERY COLUMN AT ONCE

With the table in one array, a reduction takes `axis=0` and returns one value per column. The axis named is the one that disappears, so a `(9000, 7)` table reduces to `(7,)`.

```
table.mean(axis=0)              # (9000, 7) -> (7,)  [nan nan 26.41 nan 2.12 1.16 1.15]
np.nanmean(table, axis=0)[:4]   # -> [   43.14 32102.02    26.41    58.73]
np.nanmin(table, axis=0)[:4]    # -> [   18.    5600.49     0.       9.7 ]
np.nanmax(table, axis=0)[:4]    # -> [   84.  148025.97   174.     100.  ]
```

The gaps still have to be skipped, and now on every column in one call: three of the four numeric columns have a NaN in them, so the plain `mean` returns NaN in three of the first four positions. Pair the result with the column names to read it.

```
dict(zip(COLUMNS, map(float, np.nanmean(table, axis=0))))
```

`map(float, ...)` is there because NumPy prints its own scalar types in full, as `np.float32(43.14)` rather than `43.14`.

Counting works the same way, and `axis` chooses what is being counted.

```
missing = np.isnan(table)                              # (9000, 7) bool

np.count_nonzero(missing, axis=0)                      # -> [542 823   0 361   0   0   0]  per
column
np.count_nonzero(np.any(missing, axis=1))              # -> 1622   rows with a gap anywhere
np.where(np.any(missing, axis=0))[0]                   # -> [0 1 3]  which columns have gaps
```

The middle line is the listwise count of section 5 in one expression: 1622 of 9000 rows have a gap somewhere, which is why complete-case analysis keeps only 7378.

SORTING, WITH GAPS

`np.sort` returns a sorted copy and `np.argsort` returns the order rather than the values. On a column with gaps both put NaN at the end, deliberately and consistently, which makes the smallest values easy to reach and the largest ones a trap.

```
inc = table[:, 1]                # income, 823 of the 9000 missing

np.sort(inc)[:3]                 # -> [5600.49 6104.75 6459.16]   the three smallest
np.sort(inc)[-3:]                # -> [nan nan nan]               not the three largest
```

Two ways round it. Drop the gaps before sorting, or send them to the other end.

```
present = inc[~np.isnan(inc)]            # (9000,) -> (8177,)
np.sort(present)[-3:]                    # -> [143798.83 146627.45 148025.97]

filled = inc.copy()
filled[np.isnan(filled)] = -np.inf       # NaN sorts last, -inf sorts first
np.sort(filled)[-3:]                     # -> [143798.83 146627.45 148025.97]
```

`-np.inf` is negative infinity, a float that compares below every real number, so the gaps sort to the front instead of the back.

The first is clearer. The second keeps the array's length, which matters when the positions are what you want rather than the values.

`np.argsort` answers "which rows", so one column can be ordered and another read off it. Dropping the gaps breaks that, because every index after a dropped row shifts.

```
order = np.argsort(inc)                  # (9000,) row numbers, poorest first
inc[order[-3:]]                          # -> [nan nan nan]   the same trap

n_missing = int(np.isnan(inc).sum())     # 823
top3 = order[-n_missing - 3 : -n_missing]
inc[top3]                                # -> [143798.83 146627.45 148025.97]
table[top3, 0]                           # -> [59. 83. 72.]   the ages of those three

# or keep the original row numbers through the filtered sort
np.flatnonzero(~np.isnan(inc))[np.argsort(present)][-3:]    # the same three rows
```

`np.sort` also takes an axis, and on a table it sorts every column independently. The result is useful for reading off the extremes of each column, and its rows are no longer customers: the smallest age and the smallest income almost certainly belong to two different people.

```
np.sort(table, axis=0)[:3]       # (9000, 7) -> (3, 7)  the three smallest of each column
```

QUANTILES

`np.quantile(s, q)` returns the value below which the fraction `q` of the data falls. The three at 0.25, 0.5 and 0.75 are the **quartiles**: a quarter of the scores sit below 49.8, half below 58.8, and three quarters below 67.7. The middle one is the median by definition. When the fraction does not land exactly on an observation NumPy interpolates between the two either side of it, which is why a quartile is usually not a value that appears in the data. `np.percentile` is the same function on a 0 to 100 scale.

The distance between the outer two is the **interquartile range**, 17.9 here. It describes the spread of the middle half of the data and never refers to the mean, which is what makes it hold still in the next section while the standard deviation does not.

MEAN AGAINST MEDIAN

They answer different questions, and the gap between them describes the shape of the column. On a symmetric column they nearly agree, as `score` does above. On a skewed one they do not:

```
inc = income[~np.isnan(income)]     # (9000,) -> (8177,)

inc.mean()              # -> 32102.02
np.median(inc)          # -> 28117.74   nearly 4000 lower
```

Income has a long right tail: most values sit low and a few sit far above. The mean is pulled towards the tail because it weighs every value by its size; the median is not, because it only counts how many values lie on each side. Mean above median is the signature of a right tail, and reporting only the mean hides it.

DDOF

`np.std` divides by `n`. Many statistics texts and `statistics.stdev` divide by `n - 1`, which corrects the bias you get when the data is a sample from a larger population: the deviations are measured from the sample mean, which sits exactly where it makes them as small as possible, so dividing by `n` understates the spread by a factor of `(n - 1) / n` every time. NumPy exposes the choice as `ddof`, the delta degrees of freedom, and defaults it to 0.

```
s.std()                 # -> 13.2042   ddof=0, divides by n
s.std(ddof=1)           # -> 13.2049   divides by n - 1
```

Which one to use follows from what the rows are. The question here is how spread out *these* 8639 scores are, which is a statement about the rows in hand, so `ddof=0` answers it exactly and NumPy's default is the right one. `ddof=1` is for the other question: the rows are a sample, and the number is an estimate of the spread of some larger population you cannot measure.

The size of the gap says when the choice is worth any thought at all. Here it is 13.2042 against 13.2049, the fourth digit. At twenty values it is 2.6 per cent, and at five it is 12 per cent. Above a few hundred rows the two answers are the same number for any practical purpose; below a few dozen they are not.

ROBUSTNESS TO ONE EXTREME VALUE

One absurd value, a typing error of an extra digit, moves the five statistics by wildly different amounts. This is the reason to compute more than one of them.

```
t = np.append(s, 10_000.0)          # (8639,) -> (8640,) one impossible score

t.mean() - s.mean()     # -> +1.15     the mean moves
np.median(t) - np.median(s)     # -> 0.0    the median does not
t.max() - s.max()       # -> +9900.0   the maximum is the error itself
t.std() - s.std()       # -> +94.6     13.2 becomes 107.8
```

The median and the quartiles are **robust**: they depend on the order of the values, not their size, so a single wild value moves them by at most one position. The mean, the standard deviation and the extremes are not. An impossible value usually shows up in `min` or `max`, so printing those first is the shortest check on a new column.

05

Pairwise and listwise deletion

Leaving a missing value out of a calculation can mean dropping the whole row or dropping only that one value. The two give different numbers.

Listwise deletion, also called complete-case analysis, drops every row that has a gap in any column and computes everything on what is left. **Pairwise deletion**, also called available-case analysis, keeps for each statistic every row where the columns that statistic needs are present.

```
age, inc_c = column("age"), column("income")
score_c, ten = column("score"), column("tenure_months")

complete = ~np.isnan(age) & ~np.isnan(inc_c) & ~np.isnan(score_c) & ~np.isnan(ten)
complete.sum()                      # -> 7378   of 9000 rows survive listwise deletion

score_c[~np.isnan(score_c)].mean()  # -> 58.7257   pairwise: 8639 values
score_c[complete].mean()            # -> 58.7841   listwise: 7378 values
```

The first is the mean of every score that is present. The second is the mean score among customers whose record is complete in all four columns, which is a different population, and not the one the question named. It also discards 1261 scores that were present, because something unrelated was missing in the same row.

The cost compounds with the number of columns. Four columns with a few per cent missing each cost 18 per cent of the rows between them, and the rows lost are not a random sample: a customer missing an income is more likely to be missing an age too, so listwise deletion removes a particular kind of customer from every statistic at once.

FUNCTIONS THAT IGNORE NANS

NumPy ships a parallel set of reductions that skip NaN instead of propagating it. They are exactly pairwise deletion, written shorter.

PROPAGATES NAN	SKIPS NAN	RESULT
`v.sum()`	`np.nansum(v)`	Total of the present values
`v.mean()`	`np.nanmean(v)`	Mean of the present values
`np.median(v)`	`np.nanmedian(v)`	Median of the present values
`v.std()`	`np.nanstd(v)`	Spread of the present values
`v.min()`, `v.max()`	`np.nanmin(v)`, `np.nanmax(v)`	Extremes of the present values

```
np.nanmean(score_c)     # -> 58.7257   same as masking by hand
np.nanmean(score_c) == score_c[~np.isnan(score_c)].mean()   # -> True
```

`np.nanmean` of a column that is entirely missing returns NaN and raises a `RuntimeWarning`, rather than raising an error. And there is no such function for a count, because what you want is the count of present values: `(~np.isnan(v)).sum()`, which is also what every one of these functions divided by.

06

Grouped summaries

The average score in each region. The average income on each plan. One number per category instead of one number for the whole column.

Asking it of the whole column gives `58.73`, the mean score of every customer. Asking it region by region gives six numbers, and they turn out to run from 51 to 67, which the single number hid completely.

Each of those six is an ordinary mean over a subset of the rows, and a boolean mask is what picks the subset. `region == "Upland"` is True on the rows from Upland; `~np.isnan(score_c)` is True on the rows that have a score; `&` keeps the rows where both hold. Indexing the score column with that mask gives the scores to average.

```
region = np.array([r["region"] for r in records])
score_c = column("score")
REGIONS = ["North", "Harbour", "Midland", "Eastfield", "Southgate", "Upland"]

for name in REGIONS:
    sel = (region == name) & ~np.isnan(score_c)
    print(f"{name:<10} n={sel.sum():>5}  mean score {score_c[sel].mean():6.2f}")

# North      n= 1924  mean score  50.99
# Harbour    n= 1555  mean score  61.21
# Midland    n= 1740  mean score  58.89
# Eastfield  n= 1197  mean score  64.99
# Southgate  n= 1369  mean score  55.65
# Upland     n=  854  mean score  67.46
```

Print the group size next to every group statistic. A mean over 854 rows and a mean over 1924 rows are not equally trustworthy, and a group that has quietly become empty gives a NaN that is otherwise hard to explain.

DELETION INSIDE A GROUP

The mask above tests the column being averaged. Testing completeness across all columns instead changes every group, and it changes them unevenly, because the gaps are not spread evenly across regions. The rule is the same as section 5: skip on the column the statistic is about.

```
sel_pairwise = (region == "Upland") & ~np.isnan(score_c)
sel_listwise = (region == "Upland") & complete

score_c[sel_pairwise].mean()     # -> 67.46   over 854 rows
score_c[sel_listwise].mean()     # -> 67.29   over 726 rows
```

07

Bar charts

Four counts in a column of text have to be compared by reading them. The same four as bars are compared by looking. `plt.bar` draws them in three lines, and the rest of the work is labelling.

```
names = CATEGORIES["plan"]
counts = [int((plan == name).sum()) for name in names]

plt.figure(figsize=(7, 3))
plt.bar(range(len(names)), counts, color="#39a0a8")
plt.xticks(range(len(names)), names, rotation=30, ha="right")
plt.ylabel("rows")
plt.title("rows per plan")
plt.tight_layout()
plt.show()
```

`plt.bar` places bars at the x positions it is given, so the positions are `range(len(names))` and the names are attached afterwards with `plt.xticks`. Rotating long labels and anchoring them at their right end (`ha="right"`) keeps them from overlapping. `plt.tight_layout()` resizes the axes so the rotated labels are inside the figure rather than cut off at the edge.

AXIS LABELS AND COUNTS

- **A labelled y axis.** Counts, percentages and thousands of euros all look the same without one.
- **Every category, including the empty ones.** A bar chart built from `np.unique` output omits the categories with no rows, and an absent bar is the most easily missed result there is.
- **The number of rows the figure is drawn from.** If gaps were skipped, the figure describes the present values, and the count of those belongs in the title or the caption.

A bar chart is for a categorical column, where the bars are separate things in a chosen order. A continuous column needs its values grouped into intervals first, which is a histogram and a different decision.

08

Loop forms and array equivalents

The counting and summarising loops, written as array expressions over one column. Each right-hand column skips missing values rather than propagating them.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Rows in one category	`(plan == "premium").sum()`
Rows in every category, in a chosen order	`[(plan == n).sum() for n in CATEGORIES["plan"]]`
Distinct categories and their counts	`np.unique(plan, return_counts=True)`
How many values are missing	`np.isnan(v).sum()`
How many are present	`(~np.isnan(v)).sum()`
Mean, skipping the gaps	`np.nanmean(v)`
Median and spread, skipping the gaps	`np.nanmedian(v)`, `np.nanstd(v)`
Extremes, skipping the gaps	`np.nanmin(v)`, `np.nanmax(v)`
Quartiles	`np.quantile(v[~np.isnan(v)], [.25, .5, .75])`
Mean within one group	`v[(cat == name) & ~np.isnan(v)].mean()`

TREATMENTS COMPARED

TREATMENT	WHAT IT COMPUTES	WHEN IT IS RIGHT
Leave the NaN in	NaN	Always, as a check that you noticed
Replace with zero	A different, smaller mean	Only when absent genuinely means zero
Drop rows with any gap	A statistic about complete records	When a row has to be whole, as in model input
Skip pairwise	A statistic about the present values	Summarising one column, which is everything above

FURTHER READING

- The csv module, for `DictReader`, dialects and quoting.
- Statistics routines, which lists the NaN-ignoring function beside each ordinary one.
- isnan and unique.
- pyplot.bar, for the arguments the example above does not use.

SUMMARY

A table becomes one array per column, because one array holds one dtype. A categorical column is described by counting its categories against a fixed list, so a category with no rows still appears. A numeric column needs more than its mean: the median and the quartiles survive one absurd value where the mean, the standard deviation and the extremes do not. A grouped summary is one mask per category, reported next to the size of the group. And an empty field becomes NaN, which propagates through every reduction and has to be tested with `np.isnan` rather than `==`, so skip the gaps pairwise rather than dropping every row that has one.